Jay Rosales
October 19, 2026
CS 470 Final Reflection
Presentation: https://youtu.be/dQjGdyj1SNs

Experiences and Strengths

CS 470 helped me get a better understanding of how to move a full stack application to the cloud and make its different parts work together. For my project, I worked on a question-and-answer application where users could post questions, answer questions, update votes, and delete records. Going through this process helped me understand what happens behind the scenes when someone uses a website.

Throughout the course, I worked with Amazon S3, API Gateway, Lambda, and DynamoDB. I used S3 to host the Angular frontend, API Gateway to route requests, Lambda functions to handle different operations, and DynamoDB to store the application's data. I also updated the frontend's configuration to connect to my deployed API and configured CORS so the browser could access it. This showed me that getting an application to work involves more than writing code. The settings, permissions, and connections between services also need to be correct.

I also learned about Docker and Docker Compose. I learned that a container packages an application with its dependencies so it can run consistently in different environments. Docker Compose makes setup easier by allowing multiple containers to start together. Learning about both containers and serverless services helped me understand that there are different ways to deploy an application, and each has its own benefits and challenges.

One of my strengths as a developer is that I keep working through problems even when I do not understand the cause right away. During this project, I had to check configurations, permissions, and API connections to figure out why something was not working. I am learning to slow down, break a problem into smaller parts, and test each part instead of trying to fix everything at once.

My pharmacy experience also gives me skills I can bring into software development. I am used to following procedures, paying attention to details, and communicating when something goes wrong. Although software development is a different field, those habits are useful when testing applications and troubleshooting issues.

This course supports my goal of transitioning into a software development career. Along with my computer science degree and full stack course, it has helped prepare me for junior software developer, entry-level full stack developer, and technical support engineer roles. I

still have more to learn, especially about managing applications in production, but I now have hands-on cloud experience that I can discuss with employers.

Planning for Growth

If more people started using my application, I would first check whether it was slowing down or having more errors. Since I already have separate Lambda functions for things like questions and answers, those functions could scale based on how much they were being used. This would also make it easier to update or fix one part of the application. I would test how it handled more requests and check the service limits. For errors, I would make sure users entered the required information and received a clear message if something went wrong. I would also use CloudWatch to help figure out what caused the problem and limit retries so the application did not keep repeating a failed request.

To figure out costs, I would use AWS Pricing Calculator to estimate what I might spend with different amounts of traffic. I would include requests, Lambda processing time and memory, database usage, storage, and data transfer. Then I would check the actual charges and set budget alerts. Containers with a fixed amount of resources would usually be easier to budget for because I would know what was running and for how long. However, I could still be paying for resources when people were not using the application. Serverless costs would depend more on usage, so a busy month could cost more than expected.

For now, I would stay with serverless because it means less server maintenance and can handle changes in demand. Some downsides are possible delays when functions start, service limits, and relying on AWS-specific services. Containers would give me more control over the environment, but I would have more to manage. Microservices could make separate features easier to update and scale, but having too many could also make troubleshooting more complicated. I would want a clear reason to add more services.

Elasticity would let the application adjust its capacity when more or fewer people were using it. Pay-for-service would help because much of the cost would be based on actual usage. That would be useful while I was still figuring out how much traffic to expect. I would continue checking performance and costs as the application grew to decide whether serverless was still the best option.